

Automated Root-Cause **Analyst**

From alert to answer in seconds — every number computed by ClickHouse, every conclusion proven by a trace.

A metric dips at 2 am. Someone senior loses the morning to “which segment?”

Six dashboards, one afternoon, and an answer that arrives after the damage is done.

And when teams bolt an LLM onto this, it answers fast — and **makes numbers up**.

Speed you can't trust is worse than slow.

ClickHouse computes every number.

The LLM never does arithmetic — it narrates.

A fixed, deterministic query sequence decides every drill-down.

No LLM-generated SQL, no LLM verdicts.

Every step leaves a queryable trace.

Humans may *trigger* investigations; no human — and no model — ever *authors* one.

ClickHouse Cloud — datastore · analytical engine · audit log

`dims + dictionaries → ad_events -MV→ ad_events_enriched -MV→ metrics_hourly_by_dim (rollup)`
`series_profile · incidents · investigation_steps · diagnoses`

▲ **writes (rca_rw)**

rca-mcp (custom, ~90 lines)

`list_incidents · investigate ·`
`investigate_window`

▲ **read-only (librechat_ro)**

mcp-clickhouse (official)

`follow-up SQL, SELECT-only user`

OTLP spans ►

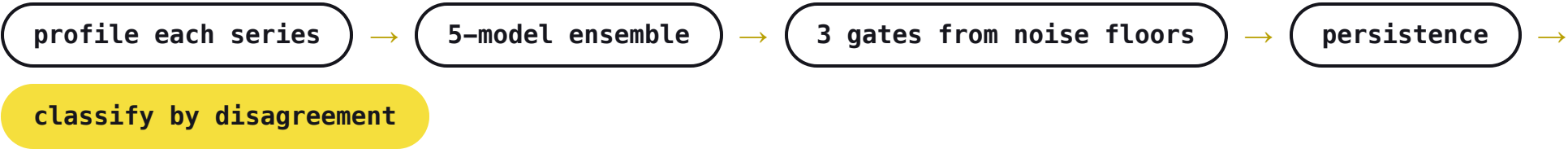
ClickStack (HyperDX)

`investigation traces · RCA`
`Overview dashboard`

LibreChat — “what incidents?” → “why did fill drop?” → “which apps were hit?”

One database is the datastore, the analytical engine, and the audit log.

Five baseline models scored in SQL, classified by disagreement



Per-series profiling (seriesPeriodDetectFFT, ACF, STL strength) picks each series’ model; window functions and robust sigmas do the scoring. The seasonality-blind model exists to be *wrong* — so weekend dips become recorded ruled_out_seasonal verdicts, not pages and not silence.

19	6	10	3
INCIDENTS FILED (DEV DATA)	REAL ANOMALIES	SEASONAL — CHECKED & RULED OUT	DISMISSED WITH EVIDENCE

Decompose the revenue identity → **sweep** dimensions weighted by contribution → **eliminate confounders** (re-sweep with the candidate excluded — residuals collapsing is printed proof the rest were shadows) → **Kitagawa mix-shift gate** before any GLOBAL verdict → **peer comparison** when history doesn't exist.

```
jun 23-25 · global fill_rate -3.4 pp  
→ Android 15 fill 0.79 → 0.43 · 98% of the global move  
every other dimension ruled out, residuals < 0.005 · ~10 queries, < 1 s of SQL
```

The narrator is checked **digit-for-digit** against the evidence

- One unmatched figure **blocks publication** — a deterministic template takes over, and `numbers_verified` is stored with the diagnosis.
- Hypothesis, **exact SQL**, result rows, decision — one row per step in `investigation_steps`, mirrored live to **ClickStack**: a span per step, the narrator call with token usage, the verdict headline on the root span.
- Ask the chat *“how do you know?”* — and it shows you the trace.

The unseen data, diagnosed with zero code changes

- **Fill-rate trench, Jul 8–9 → iOS 17.5** — 0.795 → 0.477 in-segment (–32 pp), every other OS version flat; recovered Jul 10.
- **Format price swap, Jul 9** — video –30% / rewarded +25%, clean step, other formats flat.
- **And the rare one:** boundary alerts caused by the organisers' regenerated dimension tables were **flagged as relabeling artifacts** instead of confidently mis-diagnosed. Knowing when *not* to trust a segment baseline is also analysis.

- Materialized-view cascade enriches at insert; dictionaries as the OLTP seam.
- **SummingMergeTree long-format rollup** — additive counters regroup *exactly* from hour to any window; alert numbers always reconcile with investigator numbers.
- STL / FFT / quantile models **in SQL** — detection cost scales with series × hours, not events.
- ReplacingMergeTree + deterministic ids = **idempotent re-runs**.
- A “metric” is any (numerator, denominator) pair — new domains are **whitelist rows, not redesigns**.

From alert to answer in seconds — **provable.**

Next: wire detection to a scheduler and alert webhooks · per-tenant scopes · latency-percentile metrics via quantile-sketch rollups.

Repo, hosted demo, and video — links in the README.